Cursor Agent Best Practices

Official Cursor Team Guide · January 2026
Key takeaways for your workshop sessions

Source: cursor.com/blog/agent-best-practices · Jan 2026

01

Understanding the Agent Harness

The Agent Harness: 3 Components

- Instructions — system prompt + rules that guide agent behavior
- Tools — file editing, codebase search, terminal execution
- User messages — your prompts and follow-ups that direct the work
- Cursor tunes harness per model based on internal evals & benchmarks
- ⚠️ Different models respond differently to the same prompt — harness handles this for you

💡 *You don't need to switch prompts when models change — Cursor adapts the harness automatically*

02

Start with Plans

Plan Mode: Shift+Tab Before Coding

- 1. Agent researches codebase → finds relevant files
- 2. Asks clarifying questions about requirements
- 3. Creates detailed implementation plan with file paths + code refs
- 4. Waits for your approval before building
- Plans open as editable Markdown — remove steps, adjust approach, add missing context
- Save to .cursor/plans/ → team documentation + easy resume

💡 *UChicago study: experienced devs plan more. Plan mode enforces this at scale.*

When to Start Over From a Plan

- Agent built something that doesn't match intent? Don't fix via follow-ups
- Revert all changes → refine the plan → run again
- Faster than patching in-progress agent runs
- Produces cleaner, more intentional results
- ⚠️ Chasing partial agent output is a time sink — the plan is your anchor

💡 *Treat plans like contracts: if the output violates the spec, fix the spec*

03

Managing Context

Context: What to Include vs. Let the Agent Find

Let the agent find it

- Agent uses grep + semantic search on demand
- "The authentication flow" → agent finds relevant files
- No need to tag every file in your prompt
- Including irrelevant files can confuse what matters
- @Branch gives context on current work branch

Tag it yourself when…

- You know the exact file path already
- Highly specific internal module not discoverable by name
- Cross-repo references the agent may not traverse
- Security-sensitive files you want to scope explicitly
- Agent keeps missing the same file repeatedly

When to Start a New Conversation

Start fresh

- Moving to a different task or feature
- Agent confused / repeating same mistakes
- Finished one logical unit of work
- Long conv → context noise → agent drifts to unrelated tasks
- Effectiveness visibly dropping

Continue the conversation

- Iterating on the same feature
- Agent needs context from earlier in discussion
- Debugging something it just built
- Plan still in scope, no pivot
- Use @Past Chats to import earlier context selectively

04

Extending the Agent: Rules & Skills

Rules: Always-On Static Context (.cursor/rules/)

.cursor/rules/project.md

```
# Commands
- npm run build     # Build project
- npm run typecheck # Run type checker
- npm run test      # Run tests (prefer single file for speed)

# Code style
- Use ES modules (import/export), not CommonJS (require)
- See components/Button.tsx for canonical component structure

# Workflow
- Always typecheck after a series of code changes
- API routes go in app/api/ following existing patterns
```

💡 *Commit rules to git. When agent makes a mistake, update the rule. Tag @cursor on a PR to auto-update.*

Rules: What to Include vs. What to Avoid

- ✅ Commands to run (build, test, lint, typecheck)
- ✅ Code patterns to follow — reference canonical file, don't copy it
- ✅ Architectural conventions (where routes live, module structure)
- ⚠️ Don't copy entire style guides — use a linter instead
- ⚠️ Don't document every possible command — agent knows common tools
- ⚠️ Don't add instructions for edge cases that rarely apply
- Start simple — add rules only when agent makes the same mistake repeatedly

💡 *Over-engineered rules are as harmful as missing rules — they pollute the context window*

Skills: Dynamic Capabilities (Nightly Channel)

- Defined in SKILL.md files — loaded dynamically when relevant, not always
- Unlike Rules (always included), Skills keep context window clean
- Can include: custom /commands, hooks, domain knowledge
- Hooks: scripts that run before or after agent actions (.cursor/hooks.json)
- Powerful pattern: long-running agent loop that iterates until tests pass
- Skills can integrate with secrets managers, security tools, observability platforms

💡 *Skills = dynamic context injection. Rules = permanent system context. Know when to use each.*

Long-Running Agent Loop via Hooks

.cursor/hooks/grind.ts

```
// .cursor/hooks.json
{ "hooks": { "stop": [{ "command": "bun run .cursor/hooks/grind.ts" }] } }

// grind.ts — continues agent until scratchpad says DONE
const input = await Bun.stdin.json();  // { loop_count, status }
const MAX_ITERATIONS = 5;

if (input.status !== 'completed' || input.loop_count >= MAX_ITERATIONS)
  return console.log(JSON.stringify({}));  // stop

const done = readFileSync('.cursor/scratchpad.md').includes('DONE');
if (!done) console.log(JSON.stringify({
  followup_message: `[Iter ${input.loop_count+1}] Continue. Mark DONE when complete.`
}));
```

💡 *Use for: run until tests pass · iterate UI until it matches mockup · any goal-oriented verifiable task*

05

Common Workflows

TDD with Agents: The 5-Step Protocol

- 1. Ask agent to write tests based on expected input/output — be explicit: TDD mode
- 2. Tell agent to run tests and confirm they FAIL — no implementation yet
- 3. Commit the tests when satisfied
- 4. Ask agent to write code that passes tests — no modifying tests allowed
- 5. Commit implementation once all tests pass
- ⚠️ Agents perform best with a verifiable target — tests are the feedback loop

💡 *Without a clear success criterion, the agent optimizes for output, not correctness*

Git Workflows: Custom /commands

.cursor/commands/pr.md

```
Create a pull request for the current changes.

1. Look at staged and unstaged changes with: git diff
2. Write a clear commit message based on what changed
3. Commit and push to the current branch
4. Use: gh pr create  with title/description
5. Return the PR URL when done

# Other useful commands:
- /fix-issue [n]  : gh issue view → find code → fix → open PR
- /review         : Run linters, check common issues, summarize
- /update-deps    : Update deps one by one, run tests after each
```

💡 *Store commands in .cursor/commands/ — commit to git so the whole team benefits*

06

Reviewing AI-Generated Code

3 Layers of Code Review

- 1. During generation — watch the diff view; click Stop if agent drifts
- 2. Agent review — after finish: Review → Find Issues → line-by-line analysis
- 3. Bugbot — automated PR review on every push; catches issues before human review
- Source Control tab → Agent Review → compare full branch vs main
- Ask agent for architecture diagrams (Mermaid) to reveal structural issues early
- ⚠️ AI code can look right while being subtly wrong — review speed matters

💡 *The faster the agent works, the more critical your review discipline becomes*

07

Parallel Agents & Cloud Agents

Running Agents in Parallel

Git worktrees (native)

- Each agent runs in isolated worktree
- No file conflicts between parallel runs
- Agents can build and test independently
- Click Apply to merge back to branch
- Best for: same problem, multiple approaches

Multi-model judging

- Run same prompt across multiple models simultaneously
- Compare results side-by-side
- Cursor recommends the best solution
- Exposes edge cases one model might miss
- Especially valuable for hard architectural decisions

Cloud Agents: Async Task Delegation

- Best for tasks you'd otherwise add to a todo list while working on something else
- Use cases: bug fixes, refactors, test generation, docs updates
- Start from cursor.com/agents, the editor, or your phone
- Agent clones repo → creates branch → opens PR → notifies you (Slack/email)
- You review and merge — no babysitting required
- Trigger from Slack: @Cursor [task description]
- ⚠️ Cloud agents run in remote sandboxes — you can close your laptop

💡 *Think of cloud agents as async junior engineers: good briefs = good PRs*

08

Debug Mode for Tricky Bugs

Debug Mode: Evidence-Based Bug Fixing

- 1. Generates multiple hypotheses about root cause
- 2. Instruments code with logging statements
- 3. Asks you to reproduce the bug — collects runtime data
- 4. Analyzes actual behavior to pinpoint root cause
- 5. Makes targeted fixes based on evidence — not guesses
- Best for: race conditions, timing issues, perf/memory problems, regressions
- ⚠️ Key: provide detailed reproduction steps — specificity drives quality instrumentation

💡 *Debug Mode = scientific method applied to bugs. Standard agent = educated guessing.*

5 Traits of Developers Who Get the Most from Agents

- 1. Write specific prompts — "add tests for auth.ts covering logout, using __tests__/ patterns, no mocks"
- 2. Iterate on setup — add rules/commands only after patterns emerge, not upfront
- 3. Review carefully — AI code can look right while being subtly wrong; diff every change
- 4. Provide verifiable goals — typed languages, linters, tests give agents clear success signals
- 5. Treat agents as capable collaborators — ask for plans, push back, request explanations

💡 *The constraint is almost never the model. It's the quality of context and the clarity of the goal.*

Key Takeaways

Harness

Cursor tunes agent per model — focus on context quality, not prompt hacks

Skills

Dynamic hooks for long-running loops. Hooks integrate with external tools

Plan First

Shift+Tab → Plan Mode before any significant task. Save plans to .cursor/plans/

TDD

Write tests → confirm fail → write code → iterate to pass. Never skip the red phase

Context

Let agent find files; tag only what you know. Start new conv when effectiveness drops

Review

3 layers: live diff watch → Agent review → Bugbot on PR. Speed = more review rigor

Rules

Minimal, committed to git, updated when agent repeats mistakes. Reference > copy

Parallel

Worktrees for isolation. Multi-model for hard problems. Cloud agents for async work